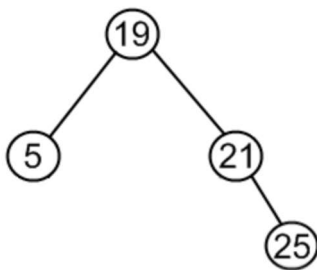


Reconstruct a Tree with its In-order and Post-order Traversal Results

To Turn in: please put the questions and your answers below them in a pdf file for the question ONE.

Background

Theory shows that the sequences of in-order traversal and post-order traversal determine a unique binary tree (this is a general **binary tree and NOT limited to a BST**). For example, given the sequence of the in-order traversal {5, 19, 21, 25} for a binary tree and given the sequence of post-order traversal {5, 25, 21, 19} for the same tree, you can reconstruct the binary tree with its graphical representation shown as below,



Question (1) to solve (20%)

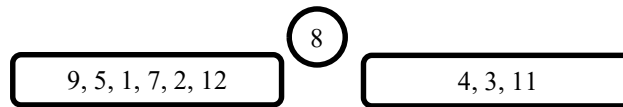
Given the sequence of in-order traversal for a **general binary tree** {9, 5, 1, 7, 2, 12, 8, 4, 3, 11}, and given the sequence of post-order traversal for the same tree {9, 1, 2, 12, 7, 5, 3, 11, 4, 8}, please explain how to reconstruct the binary tree and draw its graphical representation in your answer. Please include step by step explanations to guarantee partial credits. Note that this tree is a general binary tree, but not a BST.

Hint: thinking about the definition of in-order and post-order traversal. For in-order traversal, after its left subtree is visited, then the root is visited. This is true for any subtrees of any node. For post-order traversal, the root is always outputted at last, after its left subtree and right subtree are both visited. This is true for any subtrees of any node in the tree.

Answer (1)

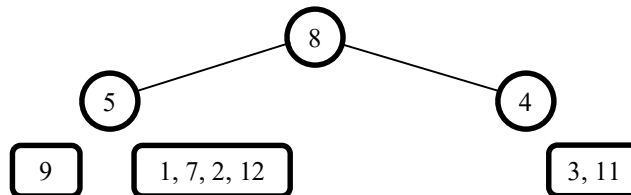
In-order traversal: {9, 5, 1, 7, 2, 12, 8, 4, 3, 11}; Post-order traversal: {9, 1, 2, 12, 7, 5, 3, 11, 4, 8}

The first step is to determine the root of the tree and break it into sections. To determine the root of the tree, look at the last element listed in the post order traversal. In this example, it is 8. Find where the root is located in the in-order traversal of the tree. Anything to the left is in the left half of the tree and anything to the right is in the right half of the tree.

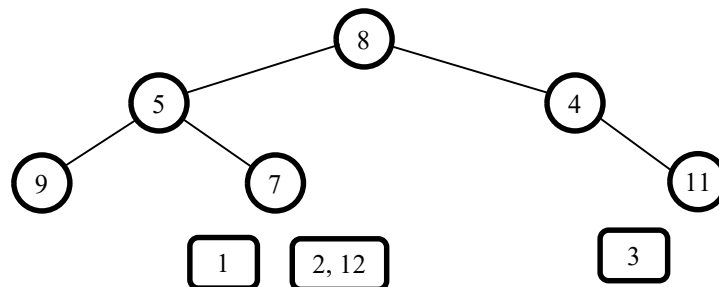


From here, we are essentially just going to repeat that same process over and over for each subsection of the tree until the tree is complete.

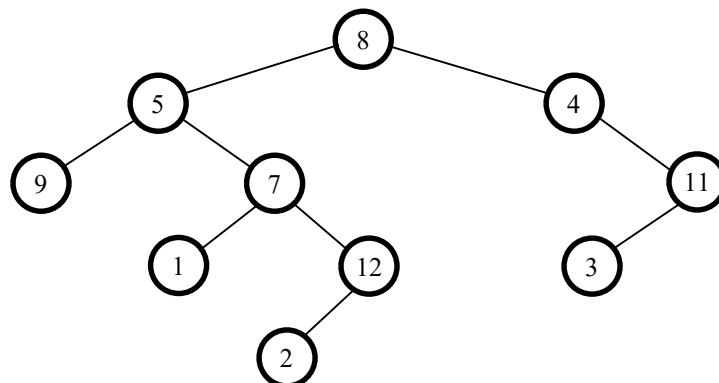
So, for each half of the tree, determine the root of the left half by finding the last element of the left half in the post order traversal. Do the same for the right half. Then split each half into left and right subtrees based on the location of the root in their respective in order traversals.



Next, we look at the subtrees. The single element doesn't need to be broken down any further, but we'll repeat the process for the rest, finding the root and splitting into left and right subtrees.



Finally, repeat the process, and we arrive at our completed tree.



Question (2) to solve (80%)

Given the sequence of in-order traversal for a general binary tree, stored in an array **inOrderSequence[]**= {9, 5, 1, 7, 2, 12, 8, 4, 3, 11 }. And given the sequence of post-order traversal for the same tree, stored in an array **postOrderSequence[]**= { 9, 1, 2, 12, 7, 5, 3, 11, 4, 8}. Please implement a **method** with its signature shown in the below,

static BinaryTree buildTree(Object inOrderSequence[], Object postOrderSequence[])

-The method buildTree() returns a BinaryTree object in memory that is constructed on the basis of the two input sequences — inOrderSequence and postOrderSequence. **Helper methods for buildTree() are allowed in your implementation.**

-Please also implement the BinaryTree class and its nested Node class, in order to test your program. Specifically, please also implement the **in-orderTraversal()** and **post-orderTraversal()** method in your BinaryTree class to traverse the tree.

-Write a Tester class, in which you will call the method, for example, **BinaryTree mytree = BinaryTree.buildTree(Object inOrderSequence[], Object postOrderSequence[])**.

-In the Tester class, then you call mytree.in-orderTraversal() and mytree.post-orderTraversal() methods to verify whether the tree returned by the buildTree() method is correct. If correct, you will see the output of mytree.in-orderTraversal() matches the input sequence **inOrderSequence[]** and the output of mytree.post-orderTraversal() matches the input sequence **postOrderSequence[]**. Please explicitly print out those input sequences and output sequences on the standard out for the grader to check.